Laptop Rentals

AlgoExpert · Hard · Heaps

The problem

- Given [start, end) rental intervals, minimum number of laptops needed so nobody waits.

Intuition

- The number of laptops needed equals the maximum number of overlapping rentals.

Brute force vs optimal

- Check overlaps for every pair → $O(n^2)$
- Sort by start + min-heap of end times → $O(n \log n)$
- Or sweep sorted starts/ends

Approach

- 1. Sort starts and ends separately
- 2. Sweep with two pointers
- 3. A start before the next end needs a laptop
- 4. Otherwise reuse one

Worked example



laptop free times (min-heap)




```
reuse a laptop → 1 laptops
```

[0, 2]: reuse

Worked example



laptop free times (min-heap)



need a new laptop → 2 laptops

[0, 4]: new laptop

Worked example



laptop free times (min-heap)



need a new laptop → 3 laptops

[1, 4]: new laptop

Worked example



laptop free times (min-heap)




```
reuse a laptop → 3 laptops
```

[4, 6]: reuse

Worked example



laptop free times (min-heap)



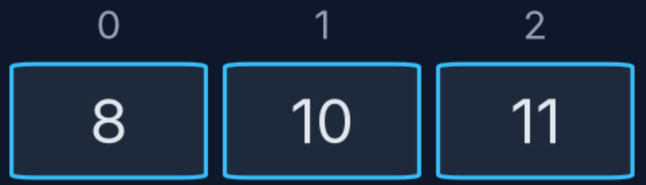

```
reuse a laptop → 3 laptops
```

[7, 8]: reuse

Worked example



laptop free times (min-heap)




```
reuse a laptop → 3 laptops
```

[9, 11]: reuse

Java solution

```java
public class LaptopRentals {
    public static int laptopRentals(int[][] times) {
        int[][] t = times.clone();
        Arrays.sort(t, (a, b) -> Integer.compare(a[0], b[0]));
        PriorityQueue<Integer> ends = new PriorityQueue<>();
        for (int[] iv : t) {
            if (!ends.isEmpty() && ends.peek() <= iv[0]) ends.poll();
            ends.add(iv[1]);
        }
        return ends.size();
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Edge cases & pitfalls

- End == start can reuse
- Sort by start time
- Empty → 0

Interview tips & follow-ups

- Same as Meeting Rooms II

Recap

- The number of laptops needed equals the maximum number of overlapping rentals.
- Complexity: O(n log n)